

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Aryan Arun	Reg. No.:	192411157
Section / Batch:	BE CSE / 2024	Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Code of Conduct

I _____ Aryan Arun (192411157) _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Aryan Arun _____

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
----------	-----------	---------------------	----------------------	----------------------	---------------------

Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				
Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CSA0624 - Design and Analysis of Algorithms

Coding Challenge — Dynamic Programming

Name: Aryan Arun

Reg.No: 192411157

Q1. 0/1 Knapsack Optimization

Problem: A warehouse manager must select items (each with a fixed weight and profit, usable at most once) to load into a truck of fixed capacity so that total profit is maximized without exceeding the weight limit.

Approach — Dynamic Programming Formulation

Define $dp[i][w]$ = the maximum profit achievable using only the first i items, with a total weight limit of w . For each item i , there are exactly two choices: leave it out (profit stays $dp[i-1][w]$), or include it, provided its weight fits (profit becomes $values[i-1] + dp[i-1][w - weights[i-1]]$). Since each item can be used at most once, the recurrence always looks back to row $i-1$ (never reusing row i), which is precisely what distinguishes 0/1 Knapsack from the unbounded version in Q2.

Recurrence: $dp[i][w] = dp[i-1][w]$ if $weights[i-1] > w$; otherwise $dp[i][w] = \max(dp[i-1][w], values[i-1] + dp[i-1][w - weights[i-1]])$.

Python Code

```
def knapsack_01(n, values, weights, W):
    # dp[i][w] = max profit using the first i items within weight limit w
    dp = [[0] * (W + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(W + 1):
            if weights[i - 1] <= w:
                # choose the better of: skip item i-1, or take it
                dp[i][w] = max(dp[i - 1][w],
                               values[i - 1] + dp[i - 1][w - weights[i - 1]])
            else:
                dp[i][w] = dp[i - 1][w] # item doesn't fit -> must skip

    return dp[n][W]

n = 3
values = [60, 100, 120]
weights = [10, 20, 30]
W = 50
result = knapsack_01(n, values, weights, W)
print(f"n={n}, Values={values}, Weights={weights}, W={W}")
print(f"Maximum Profit = {result}")
```

Output

```
n=3, Values=[60, 100, 120], Weights=[10, 20, 30], W=50
Maximum Profit = 220
```

Verification: the optimal selection is items with weight 20 and 30 (profits 100+120=220, total weight 50, exactly at capacity) — better than any other combination, including weight 10+20=30 (profit 160) or all three items (weight 60, exceeds capacity).

Complexity Analysis

Time Complexity: $O(n \times W)$ — the DP table has $(n+1) \times (W+1)$ cells, each computed in $O(1)$ time from previously computed cells.

Space Complexity: $O(n \times W)$ for the full 2-D table; this can be optimized to $O(W)$ using a single 1-D array updated right-to-left, since each row only depends on the row directly above it.

Q2. Unbounded Knapsack

Problem: A manufacturing unit can use each item type an unlimited number of times. Maximize total profit within a given capacity, given each item's weight and profit.

Approach — Dynamic Programming Formulation

Define $dp[w]$ = the maximum profit achievable using any combination of items (each reusable any number of times) with total weight exactly at most w . For each capacity w from 1 up to W , consider every item: if it fits ($weights[i] \leq w$), it can improve $dp[w]$ by taking $values[i]$ plus the best achievable with the remaining capacity $w - weights[i]$. Crucially, $dp[w - weights[i]]$ may itself already include copies of item i — this is what allows unlimited reuse, unlike 0/1 Knapsack, which always looks at a strictly smaller item index to guarantee each item is used at most once.

Recurrence: $dp[w] = \max \text{ over all items } i \text{ with } weights[i] \leq w \text{ of } (values[i] + dp[w - weights[i]])$, with $dp[0] = 0$.

Python Code

```
def unbounded_knapsack(n, values, weights, W):
    # dp[w] = max profit achievable with capacity w, items reusable
    dp = [0] * (W + 1)

    for w in range(1, W + 1):
        for i in range(n):
            if weights[i] <= w:
                dp[w] = max(dp[w], values[i] + dp[w - weights[i]])

    return dp[W]

n = 3
values = [10, 30, 20]
weights = [5, 10, 15]
W = 100
result = unbounded_knapsack(n, values, weights, W)
print(f"n={n}, Values={values}, Weights={weights}, W={W}")
print(f"Maximum Profit = {result}")
```

Output

```
n=3, Values=[10, 30, 20], Weights=[5, 10, 15], W=100
Maximum Profit = 300
```

Verification: item 2 (weight 10, profit 30) has the best profit-to-weight ratio (3.0 per unit weight) of the three items. Using it exclusively — 10 copies at weight 10 each, exactly filling the 100-unit capacity — gives $10 \times 30 = 300$, matching the computed optimum; no other combination of the three item types reaches a higher total.

Complexity Analysis

Time Complexity: $O(n \times W)$ — for each of the W capacity values, all n items are checked once.

Space Complexity: $O(W)$ — only a single 1-D array is needed (smaller than 0/1 Knapsack's $O(n \times W)$, since there is no need to track a separate row per item — reuse means every capacity only needs one running best value).

Q3. Traveling Salesman Problem (Held-Karp)

Problem: A delivery agent must visit every city exactly once and return to the start, minimizing total travel cost, given a cost matrix between every pair of cities.

Approach — Dynamic Programming Formulation (Held-Karp)

Brute-force checking of all $(n-1)!$ possible routes is only feasible for very small n . The Held-Karp algorithm instead uses bitmask dynamic programming: define $dp[mask][i]$ = the minimum cost to start at city 0, visit exactly the set of cities represented by the bitmask 'mask', and end at city i (where i must be a member of mask). Starting from $dp[\{0\}][0] = 0$, every other state is built by extending a smaller visited-set with one new city, so each state depends only on states with fewer bits set in the mask — a smaller sub-problem, exactly the divide-and-conquer/DP principle. The final answer takes the minimum, over every city i (other than the start), of the cost to reach i having visited everyone, plus the cost to return from i back to city 0.

Recurrence: $dp[mask \mid (1 \ll v)][v] = \min(dp[mask \mid (1 \ll v)][v], dp[mask][u] + cost[u][v])$ for every u in mask and every v not in mask. Final answer = min over $i \neq 0$ of $(dp[FULL_MASK][i] + cost[i][0])$.

Python Code

```
def tsp_held_karp(n, cost):
    # dp[mask][i] = min cost to visit exactly the cities in 'mask',
    # starting at city 0 and currently ending at city i
    dp = [[float('inf')] * n for _ in range(1 << n)]
    dp[1][0] = 0 # base case: only city 0 visited, currently at city 0

    for mask in range(1 << n):
        for u in range(n):
            if not (mask & (1 << u)) or dp[mask][u] == float('inf'):
                continue
            for v in range(n):
                if mask & (1 << v):
                    continue # v already visited
                new_mask = mask | (1 << v)
                new_cost = dp[mask][u] + cost[u][v]
                if new_cost < dp[new_mask][v]:
                    dp[new_mask][v] = new_cost

    full_mask = (1 << n) - 1
    # close the tour: return from the last city back to city 0
    return min(dp[full_mask][i] + cost[i][0] for i in range(1, n))

n = 4
cost = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0],
]
result = tsp_held_karp(n, cost)
print(f"n={n}")
for row in cost:
    print(row)
print(f"Minimum Cost = {result}")
```

Output

```
n=4
[0, 10, 15, 20]
[10, 0, 35, 25]
[15, 35, 0, 30]
[20, 25, 30, 0]
```

Minimum Cost = 80

Verification: the route 0 → 1 → 3 → 2 → 0 costs 10 + 25 + 30 + 15 = 80, matching the computed minimum; this can be confirmed as optimal since Held-Karp exhaustively considers every visiting order via its bitmask states, just far more efficiently than checking each permutation individually.

Complexity Analysis

Time Complexity: $O(n^2 \times 2^n)$ — there are 2^n possible subsets (masks) and n possible 'current city' values per mask, and each state transition considers up to n possible next cities.

Space Complexity: $O(n \times 2^n)$ for the DP table.

Why this matters: $O(n^2 \times 2^n)$ is a dramatic improvement over the brute-force $O(n!)$ of checking every permutation directly — for $n=15$, $n!$ is over a trillion while $n^2 \times 2^n$ is only about 7.4 million, making Held-Karp practical for moderate n where brute force is hopeless (though it still becomes infeasible for very large n , e.g., $n > 20-25$, due to the exponential 2^n factor).

Q4. Floyd-Warshall Algorithm

Problem: A communication network has some node pairs directly connected and some not (represented as INF in the cost matrix). Compute the shortest path between every pair of nodes.

Approach — Dynamic Programming Formulation

Define $\text{dist}[i][j]$ as the current best known shortest distance from node i to node j . Floyd-Warshall improves this matrix incrementally by considering each node k , in turn, as a potential 'intermediate' waypoint on the path from i to j : if going $i \rightarrow k \rightarrow j$ is shorter than the currently known direct estimate $i \rightarrow j$, the matrix is updated. After considering all n nodes as possible intermediates, $\text{dist}[i][j]$ holds the true shortest path between every pair, since any shortest path's full sequence of intermediate nodes will have been considered by the time the loop finishes.

Recurrence: $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$ for every intermediate node k , applied for all i, j after fixing k .

Python Code

```
INF = float('inf')

def floyd_warshall(n, graph):
    dist = [row.copy() for row in graph]    # work on a copy of the input matrix
    for k in range(n):                      # try every node as an intermediate
        for i in range(n):
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
    return dist

def fmt(v):
    return "INF" if v == INF else str(v)

n = 3
graph = [
    [0, 5, INF],
    [2, 0, 3],
    [INF, INF, 0],
]
result = floyd_warshall(n, graph)
print(f"n={n}\nInput Graph:")
for row in graph:
    print([fmt(v) for v in row])
print("Shortest Path Matrix:")
for row in result:
    print([fmt(v) for v in row])
```

Output

```
n=3
Input Graph:
['0', '5', 'INF']
['2', '0', '3']
['INF', 'INF', '0']
Shortest Path Matrix:
['0', '5', '8']
['2', '0', '3']
['INF', 'INF', '0']
```

Verification: $\text{dist}[0][2]$ was directly INF (no direct edge from node 0 to node 2), but going through node 1 ($0 \rightarrow 1 \rightarrow 2$) costs $5 + 3 = 8$, which is finite and therefore becomes the new shortest distance — exactly the update shown in the output. All other entries were already optimal (no intermediate node improved them), including the entries that remain INF: node 2 has no outgoing edges in the input graph at all, so nothing reachable from node 2 can be updated, correctly leaving row 2 unchanged.

Complexity Analysis

Time Complexity: $O(n^3)$ — three nested loops (over k , i , and j), each ranging over all n nodes.

Space Complexity: $O(n^2)$ for the distance matrix (the algorithm can update it in place, as done here, without needing extra matrices per iteration of k).

Why it works for all pairs at once: unlike running a single-source shortest-path algorithm (e.g., Dijkstra) from every node individually (which would cost $O(n)$ runs $\times$ $O(\text{the per-run complexity})$), Floyd-Warshall computes all n^2 pairwise shortest distances together in one $O(n^3)$ pass, and — unlike Dijkstra — it handles negative edge weights correctly as long as there are no negative-weight cycles.

Q5. Optimal Binary Search Tree (OBST)

Problem: Given a sorted set of keys and their access frequencies, construct a Binary Search Tree that minimizes the expected total search cost (frequency-weighted depth of each key).

Approach — Dynamic Programming Formulation

Define $\text{cost}[i][j]$ = the minimum expected search cost of a BST built from $\text{keys}[i..j]$ alone. For a single key, $\text{cost}[i][i] = \text{freq}[i]$ (a lone node costs its own frequency once, at depth 1). For a range of keys, every key r in $[i, j]$ is tried as the root of that sub-tree: choosing r splits the range into a left sub-tree ($\text{keys}[i..r-1]$) and right sub-tree ($\text{keys}[r+1..j]$), and the total cost becomes the sum of the two sub-trees' optimal costs plus the sum of all frequencies in the range (because making any key the root adds exactly one extra unit of depth to every other key in the range, which is captured by adding the full frequency sum once). The best root for each range is the one minimizing this total.

Recurrence: $\text{cost}[i][j] = \min \text{ over } r \text{ in } [i, j] \text{ of } (\text{cost}[i][r-1] + \text{cost}[r+1][j] + \text{sum}(\text{freq}[i..j]))$, with $\text{cost}[i][i] = \text{freq}[i]$.

Python Code

```
def optimal_bst(n, keys, freq):
    # cost[i][j] = min expected search cost of a BST built from keys[i..j]
    cost = [[0] * n for _ in range(n)]

    for i in range(n):
        cost[i][i] = freq[i]          # base case: single-key sub-tree

    for length in range(2, n + 1):    # consider increasing range lengths
        for i in range(n - length + 1):
            j = i + length - 1
            cost[i][j] = float('inf')
            fsum = sum(freq[i:j + 1]) # every key in range gets +1 depth from the chosen
root
            for r in range(i, j + 1): # try every key as the root of this range
                left = cost[i][r - 1] if r > i else 0
                right = cost[r + 1][j] if r < j else 0
                total = left + right + fsum
                if total < cost[i][j]:
                    cost[i][j] = total

    return cost[0][n - 1]

n = 3
keys = [10, 12, 20]
freq = [34, 8, 50]
result = optimal_bst(n, keys, freq)
print(f"n={n}, Keys={keys}, Freq={freq}")
print(f"Minimum Cost = {result}")
```

Output

```
n=3, Keys=[10, 12, 20], Freq=[34, 8, 50]
Minimum Cost = 142
```

Verification: choosing key 20 (frequency 50) as the root, with key 10 as its left child and key 12 as the right child of key 10 (a right-leaning left sub-tree), gives depths of 1 for key 20, 1 for key 10... checking the DP's own construction, the optimal split places key 20 (highest frequency) closest to the root so its high-frequency lookups stay cheap, while the less-frequent keys 10 and 12 sit deeper — exactly the intuition Optimal BST is designed to exploit, and the DP confirms 142 as the minimum achievable expected cost across every possible tree shape.

Complexity Analysis

Time Complexity: $O(n^3)$ — there are $O(n^2)$ sub-ranges $[i, j]$, and for each one, up to n possible roots r are tried.

Space Complexity: $O(n^2)$ for the cost table.

Why not just use frequency to build a BST greedily: a BST built purely from key order (ignoring frequency) can place a very frequently searched key deep in the tree purely by bad luck of insertion order, inflating expected search cost; Optimal BST's DP instead searches every possible tree shape (via every possible root choice at every range) and is guaranteed to find the one minimizing frequency-weighted depth, which a greedy 'always pick the highest-frequency key as root' heuristic is not guaranteed to do once sub-tree interactions are considered.